Technische Beschreibung

Aufbau

Teil	Technik
Server	Python 3.12, FastAPI, Uvicorn; läuft als Hintergrundprozess auf einem Windows-PC
Datenbank	SQLite (eine Datei `daten/lager.db`, WAL-Modus), Zugriff über SQLAlchemy Core
Oberfläche	Server-gerenderte Seiten (Jinja2), HTMX, Alpine.js, Tailwind CSS; alle Dateien lokal, kein Internet nötig
Scanner-App	`/m`, installierbare Web-App (Manifest, Service Worker `/m/sw.js`); Scantaste über Tastatureingabe (DataWedge), Kamera-Scan über BarcodeDetector bzw. ZXing (lokal)
Offline-Modus	Warteschlange in IndexedDB des Geräts, Artikelkatalog `/m/katalog.json` zwischengespeichert, Übertragung über `/m/sync` mit UUID je Buchung (Tabelle `sync_log` verhindert Doppelbuchungen)
HTTP / HTTPS	HTTP (Port 8080) lauscht nur auf `127.0.0.1` – für den Browser am Lager-PC. Alle anderen Geräte nur über HTTPS (Port 8443). Abschaltbar mit `http_nur_lokal = false` (nicht empfohlen).
Zertifikate	Eigene kleine CA (`daten/lager_ca.crt`, Download `/zertifikat.crt`; der private Schlüssel `daten/lager_ca_schluessel.pem` wird nie ausgeliefert). Die CA ist per *Name Constraints* (kritisch) auf 10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16, 127.0.0.0/8, 169.254.0.0/16 und die Namen dieses PCs beschränkt – auch mit gestohlenem Schlüssel lassen sich keine Zertifikate für fremde Domains ausstellen. Das Serverzertifikat wird beim Start erneuert, wenn sich IP/PC-Name geändert haben oder es in weniger als 30 Tagen abläuft; öffentliche IP-Adressen werden nicht aufgenommen.
Etikettendruck	TSPL direkt an HPRT HT100 (Windows-Spooler RAW über winspool.drv oder TCP 9100); Fallback Browserdruck
Sicherung	ZIP mit konsistenter SQLite-Kopie (Backup-API) und Anhängen; mehrere Uhrzeiten am Tag, mehrere Speicherorte, nur bei Änderung, Wiederherstellen im laufenden Betrieb (`app/services/sicherung.py`)

Gleichzeitige Buchungen (PC, mehrere Handscanner, API) werden über Schreibtransaktionen mit sofortiger Sperre (`BEGIN IMMEDIATE`) nacheinander ausgeführt; Lesen blockiert nicht. Getestet mit parallelen Entnahmen des letzten Teils (Threads und echte HTTP-Anfragen).

Anmeldung: Passwörter mit mindestens 10 Zeichen, gespeichert als PBKDF2-SHA256 (240.000 Runden). Nach 5 Fehlversuchen je Konto und Gerät bzw. 20 je Gerät ist die Anmeldung 15 Minuten gesperrt (nur im Speicher; Neustart hebt die Sperre auf). Der erste Administrator kann nur am Lager-PC selbst angelegt werden. Die Sitzung liegt signiert im Cookie (30 Tage; über HTTPS mit `Secure`, immer `HttpOnly`, `SameSite=Lax`), wird aber bei jedem Aufruf gegen die Tabelle `users` geprüft – Sperre, Rollenwechsel und Passwortänderung wirken sofort. Größere Daten (Sammelentnahme-Liste, Übernahmebericht) liegen in der Tabelle `settings`, nicht im Cookie (Browsergrenze 4 KB). Weiterleitungen (`weiter`, Referer) sind auf Pfade der eigenen Anwendung beschränkt; Anhänge werden nur als Bild/PDF/Text im Browser angezeigt, alles andere als Download.

Datensicherung

Ablauf je Termin: SQLite-Backup-API kopiert die laufende Datenbank in eine temporäre Datei; Buchungen laufen dabei weiter. Daraus wird ein **Fingerabdruck** (SHA-256 über alle Tabellen und die Liste der Anhänge mit Größe und Änderungszeit) berechnet. Nicht mitgezählt werden der Sicherungsstatus in `settings` (`backup_*`, `letztes_backup`, `letzte_meldemail`) und `users.letzter_login`. Die ZIP-Datei wird einmal gebaut und in jeden Speicherort kopiert: zuerst als `….zip.teil`, mit `fsync` auf den Datenträger geschrieben, per CRC-Prüfung (`testzip`) kontrolliert und erst dann umbenannt. Bei „nur bei Änderung“ wird ein Speicherort übersprungen, wenn dort die Datei mit demselben Fingerabdruck noch vorhanden ist. Ein neuer oder geleerter Speicherort bekommt also sofort eine Sicherung.

Zustand je Speicherort (letzte erfolgreiche Prüfung, Datei, Größe, freier Platz, letzter Fehler) steht als JSON in `settings.backup_status`, der zuletzt erledigte Termin in `settings.backup_termin`. Ein fehlgeschlagener Speicherort wird alle 30 Minuten gezielt erneut versucht; die anderen werden dabei nicht neu beschrieben. Warnung auf der Startseite (nur Administratoren): letzter Versuch fehlgeschlagen, keine Sicherung vorhanden, letzte Sicherung älter als 26 Stunden, freier Platz unter dem Dreifachen der letzten Sicherung. Aufräumen: Dateien `lager_backup_*.zip` älter als die Frist, die neuesten 3 bleiben immer.

Wiederherstellen (`/einstellungen/backup-wiederherstellen`, nur Administratoren, Passwort + Bestätigung, Fehlversuche zählen zur Anmeldesperre): Prüfung der ZIP (CRC, keine Pfade außerhalb von `anhaenge/`, `PRAGMA integrity_check`, Pflichttabellen, mindestens ein aktiver Administrator) **vor** jeder Änderung → Sicherung des aktuellen Stands als `vor_wiederherstellung_….zip` → Anhänge in einen Nachbarordner entpacken → Datenbank per SQLite-Backup-API in die laufende Datenbank kopieren (WAL-Modus bleibt, keine Reste alter `-wal`-Dateien, wartende Buchungen laufen danach weiter) → Verbindungen neu öffnen, fehlende Tabellen älterer Versionen ergänzen → Anhangsordner tauschen → Eintrag im Protokoll. Der Sicherungsstatus des PCs bleibt erhalten.

Ordnerauswahl: `/einstellungen/backup-ordner` listet Laufwerke (Windows: `GetLogicalDrives`, Laufwerkstyp und -name) bzw. Unterordner ohne versteckte/Systemordner – so, wie der Prozess der Lagerverwaltung sie sieht (beim Dienst: NETZWERKDIENST).

Datenmodell

`artikel`, `lagerplaetze`, `bestand` (Menge je Artikel und Platz), `bewegungen` (jede Buchung mit Zeit, Benutzer, Menge ±, Bestand danach, Empfänger, Kostenstelle, Zweck, Quelle pc/mobil/offline/api/import), `reservierungen`, `bestellungen`, `inventuren` / `inventur_pos`, `lieferanten`, `anhaenge`, `users`, `audit`, `settings`, `sync_log`.

Sichtlagerboxen sind eigene Zeilen in `lagerplaetze` mit dem Code des Platzes plus `-B<Nummer>` (z. B. `C1-R2-3-B4`). Die Zugehörigkeit zum Platz ergibt sich aus dem Code (`queries.box_eltern`), es gibt keine zusätzliche Spalte und keine Datenbankänderung. Buchen, Scannen, Etiketten, Inventur, Sicherung und Import behandeln Boxen wie jeden Platz. Umbenennen eines Platzes benennt seine Boxen mit (auch in `bewegungen.lagerplatz`). Nachbestell-Markierung (`queries.nachbestell_plaetze`): alle Plätze mit einer Bestandszeile eines aktiven Artikels im Status „melden“ oder „kritisch“, wie unter *Nachbestellung*. Die Farbe Weinrot ist als `--wein` in `frontend/input.css` hinterlegt.

Mengen: Komma oder Punkt als Dezimaltrenner; „nan“, „inf“ und Werte über 1 Milliarde werden abgelehnt.

Buchungsarten: `eingang`, `ausgang`, `umbuchung` (zwei verknüpfte Zeilen), `inventur`, `anlage`, `ausleihe`, `rueckgabe`, `info` (nur Historie aus dem Altsystem).

Alle Bestandsänderungen laufen über `app/services/lager.py` (Klasse `Lager`).

Übernahme aus Casper

`app/services/casper_import.py` liest den HeidiSQL-Export (nur INSERT-Anweisungen am Zeilenanfang – das `INSERT` im Casper-Trigger `bewegungsdaten_after_insert` wird ignoriert) und übernimmt:

Casper	Neu
stammdaten.Artikelnummer / Bezeichnung	artikel.nummer / bezeichnung
Meldebestand / Mindestbestand (Text, 0 = nicht gepflegt)	Zahl oder leer
Freifeld1 „Lieferant“	lieferanten (Schreibvarianten zusammengeführt)
Freifeld2 „Gruppe“, Freifeld3 „Typenbezeichnung“	kategorie, typ
Freifeld4 „Austragungsgrund“, Freifeld5	notiz
lagerorte (Leerzeichen bereinigt, Duplikate summiert, leer → OHNE-PLATZ)	lagerplaetze + bestand
bewegungsdaten (Eingang, Ausgang, Umbuchung/Umlagerung, Inventur, Neuer Artikel, Artikel bearbeitet/gelöscht)	bewegungen (quelle = import)
benutzer	users (gesperrt, ohne Passwort)

Lagerort-Zeilen zu nicht mehr vorhandenen Artikeln werden nicht übernommen (im Bericht genannt).

Schnittstelle

Token-Authentifizierung (`Authorization: Bearer <Schlüssel>`, Schlüssel je Benutzer unter *Einstellungen → Schnittstelle*). Endpunkte unter `/api/v1/…`: Artikel (Liste, Detail mit Beständen je Platz), Lagerplätze, Bewegungen seit ID, Buchung anlegen, Reservierung anlegen. Beschreibung maschinenlesbar unter `/api/openapi.json`. Gedacht für die spätere Anbindung der Instandhaltungs-Software (Ersatzteile reservieren/entnehmen je Auftrag).

Entwicklung

```
pip install -r requirements.txt
cd frontend && npm install && ./build_css.sh
python -m app.main                      # http://localhost:8080 (nur lokal), https://<PC>:8443
python -m pytest tests                  # LV_CASPER_EXPORT=<datei.sql> testet zusätzlich die Übernahme echter Daten (Sollwerte 821 / 377 / 2702)
```